

dsABRE: A SABRE-Style Router for Multi-Core Distributed Quantum Computers

Sanjiang Li *

Abstract—Distributed quantum computers execute a circuit across small quantum processing units linked by entanglement. Their compiler must minimise consumption of Einstein–Podolsky–Rosen (EPR) pairs while still routing large circuits quickly and reliably on hardware with finite communication ports and finite core capacity. We present dsABRE, a distributed extension of the widely used SABRE routing principle. Rather than treating a teleport as a high-cost SWAP, dsABRE scores each inter-core teledata move as one macro-action: the EPR pair it spends, the port staging needed to make it available, the destination capacity it consumes, and the front-layer progress it provides with lookahead discounted by depth in the circuit’s directed acyclic graph (DAG). The availability and capacity terms are the two routing-state constraints that a directional teleport introduces beyond a local SWAP. A checked capacity condition and bounded recovery mechanism further ensure that, under stated preconditions, dsABRE terminates after routing every gate and does not abort at saturation. Across 21 Munich Quantum Toolkit (MQT) Bench circuits with 25–64 logical qubits, dsABRE reduces geometric-mean EPR consumption by 49–62% relative to TelesABRE. It completes all 45 evaluated circuit–architecture instances against TelesABRE’s 40, and compiles a 360-qubit QFT on twenty cores in 48s. These results show that capacity-aware, feasibility-preserving extensions of a simple SABRE-style objective can provide effective, scalable, and robust distributed routing. Code is available at <https://github.com/ebony72/dsabre>.

Index Terms—distributed quantum computing, circuit routing, qubit teleportation, SABRE, EPR minimisation, qubit mapping, lookahead heuristic

I. INTRODUCTION

Distributed quantum computers (DQCs) combine several small quantum processing units (QPUs) through entanglement links [1]. They offer a practical route to scaling beyond a single processor, but introduce a compilation problem absent from monolithic devices: a two-qubit gate whose operands occupy different cores requires inter-core communication. In this work that communication is supplied by an Einstein–Podolsky–Rosen (EPR) pair. EPR generation is slow, noisy and rate-limited relative to local operations in current hardware [2], [3], so a router should avoid spending an EPR pair unless the move it enables is worthwhile.

Distributed compilation has three connected tasks: (i) allocate logical qubits to cores, (ii) choose how to realise a non-local interaction, and (iii) route and schedule the resulting local and inter-core operations [4]. Hypergraph partitioning addresses the first task by exposing global communication structure [5], [6]. An allocation alone, however, does not

ensure that the computation can execute on a device with finite ports, finite core capacity, and non-trivial intra-core connectivity. During routing, a state may have to be moved to a communication port, that port may be occupied, and the target core may lack a spare physical slot. These are dynamic constraints of the evolving layout, not properties of the initial partition alone.

A practical routing objective. A useful compiler must be effective, efficient, and robust: it should reduce the expensive resource, compile instances at useful scale, and return a valid routed circuit. These requirements motivate a SABRE-style design. SABRE [7] is the layout and routing pass Qiskit [8] selects by default: it selects a SWAP by the progress it makes on the *front-layer* two-qubit gates, with bounded lookahead. Its appeal is not global optimality but a simple, scalable decision rule that performs well in production compilation flows. The question addressed here is what that rule must price when the machine is several QPUs rather than one.

The answer is not to replace a SWAP by a teleport on a higher-weight edge. A SWAP is symmetric and preserves the occupancy of the regions it joins. A teleport is directional: it is feasible only after its source state reaches a communication port, and it consumes a free slot in the destination core. Thus, an inter-core move has three inseparable consequences: it spends an EPR pair, may require port staging to become available, and reduces the capacity available for future moves. Treating these effects as one macro-action is the unifying design principle of dsABRE.

dsABRE is a SABRE-style teledata router that scores a candidate macro-action—source staging, port eviction if needed, and one teleport—by the routing progress it buys and the resources it consumes. The progress term is SABRE’s front-layer distance reduction plus a lookahead discounted by depth in the circuit’s directed acyclic graph (DAG), the dependency graph defined in Section II-A. The two added costs are an availability term for staging at a port and a graded capacity term for the destination core: the two routing-state constraints introduced when a one-way teleport replaces a local SWAP. Latency, coherence, link contention, and fidelity are teleport-specific too, but lie outside the model optimised here (Section VI). The capacity floor is deliberately kept out of the score, as a hard admission test applied when candidates are generated (Eq. 4): a weight can be outvoted by a large progress term, an admission test cannot. The score therefore ranks feasible progress-making moves, while the floor and the bounded recovery mechanism keep finite capacity from becoming an unrecoverable mid-route dead end (Sections III-A–III-F).

Scope of communication. This paper deliberately optimises

S. Li is with the Centre for Quantum Software and Information, University of Technology Sydney, Australia (e-mail: sanjiang.li@uts.edu.au).

*ORCID: 0000-0002-3332-2546.

teledata, which moves one logical state to a new core and consumes its EPR pair immediately, so that EPR accounting is independent of entanglement storage lifetime and the capacity-aware routing problem is isolated. Telegate can be preferable when a group of compatible remote gates shares an entangled resource, but its benefit depends on gate grouping, link-buffer occupancy, and the lifetime of retained entanglement [9]: it is a distinct joint primitive-selection problem, not an omitted zero-cost candidate. dSABRE does not claim teledata dominates telegate, and identifies burst extraction and telegate selection as extensions (Section VI).

Scope of architecture and comparison. The algorithm is defined on an arbitrary *connected* coupling graph with designated ports and queries that graph through distance tables: each core’s intra-core graph is connected and so is the core graph, so every distance the router queries is finite. Nothing further is assumed—not degree, regularity, diameter, or the number and placement of ports.

The evaluation exercises that generality over four architectures: the grid-of-grids devices of TeleSABRE and two networks of IBM heavy-hex cores, a ring and a star. The reported EPR reductions are established across all four, which differ in core degree, diameter, and port placement. Likewise, EPR counts are directly comparable only when methods use the same communication primitive, architecture, and cost accounting. TeleSABRE [10] is the primary head-to-head baseline because it shares the teledata setting and local-SWAP accounting. Comparisons with DMapS [11] and pytket-dqc are controlled cross-model studies that state the remote-SWAP or telegate assumptions, communication capacities, and entanglement-lifetime conditions needed to interpret their EPR counts.

Contributions.

- **A unified capacity-aware teledata score.** We formulate every inter-core move as a macro-action and extend SABRE’s distance-progress objective with the two routing-state costs that distinguish teleportation from a SWAP: port availability and the destination capacity it consumes. The score uses front-layer distance reduction plus DAG-depth-discounted lookahead, and exhausts executable local front-layer work before considering an EPR-consuming action.
- **Feasibility-aware routing with completion conditions.** We give pre-routing capacity conditions and a recovery transaction (Algorithm 2) for the saturated states in which the ordinary loop has no legal move that makes progress. Under the stated conditions dSABRE routes every gate and does not abort; this is a completion guarantee, not a claim of EPR optimality. The measurements locate where it earns its place: no reported route among the 21 circuits at 25–64 qubits invokes recovery at all, whereas at scale it retires 6, 10, and 66 gates over the reported 100-, 200-, and 360-qubit routes.
- **Evidence of practical effectiveness and efficiency.** Across 21 Munich Quantum Toolkit (MQT) Benchmark circuits with 25–64 logical qubits, dSABRE reduces geometric-mean EPR consumption by 49–62% relative to TeleSABRE. It completes all 45 evaluated circuit–

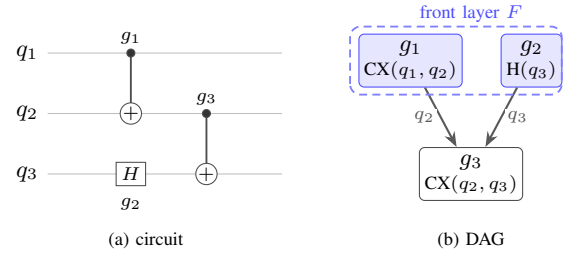


Fig. 1. DAG for a circuit with $CX(q_1, q_2)$, $H(q_3)$, and $CX(q_2, q_3)$. The first two gates have no predecessors and form the initial front layer. The final CX becomes ready only after both have executed.

architecture instances, against TeleSABRE’s 40. The implementation also compiles a 360-qubit QFT on twenty cores in 48s, with compile time growing near-linearly in gate count at fixed architecture (Section IV-C). Ablations separate the effects of capacity handling, lookahead construction, and recovery, while the cross-model experiments distinguish like-for-like comparisons from results contingent on different EPR use assumptions.

Section II gives the circuit, teleportation, architecture and SABRE background; Section III presents dSABRE and its completion argument; Section IV evaluates it; and Sections V–VI discuss related work and extensions. Supplementary material at the URL above carries the full completion proof, the complexity validation, per-circuit and per-seed counts, benchmark provenance, the parameter and cost-model sweeps, and the capacity, lifetime and action-level audits behind the cross-model comparisons.

II. BACKGROUND

A. Circuits and routing

A quantum circuit is represented as a directed acyclic graph (DAG): each node is a gate and each edge records a dependency induced by a shared logical qubit. The *front layer* is the set of dependency-free gates and can be executed in parallel when the hardware permits. This paper considers single-qubit (1Q) gates and two-qubit (2Q) gates, in particular CX gates. Together, CX and arbitrary 1Q unitaries are *exactly* universal: they decompose any n -qubit unitary [12], whereas a finite set such as $\{CX, H, T\}$ is only *approximately* so. Hardware provides a finite set: the benchmark circuits of Section IV-A are transpiled into IBM’s $\{CX, R_z, \sqrt{X}, X\}$.

Let a circuit contain n logical qubits and let the target architecture have $P \geq n$ physical qubits. An initial layout is an injective mapping from logical to physical qubits. A 2Q gate is executable only when its operands are adjacent in the coupling graph. Routing therefore inserts local SWAPs and, in a distributed machine, inter-core communication operations; each local SWAP costs c_{swap} and each inter-core teleport c_{tele} . A SWAP exchanges the states of adjacent qubits and decomposes into three CX gates. Figure 1 illustrates the DAG and front-layer notion used by the router; *circuit depth* is the length of its longest path.

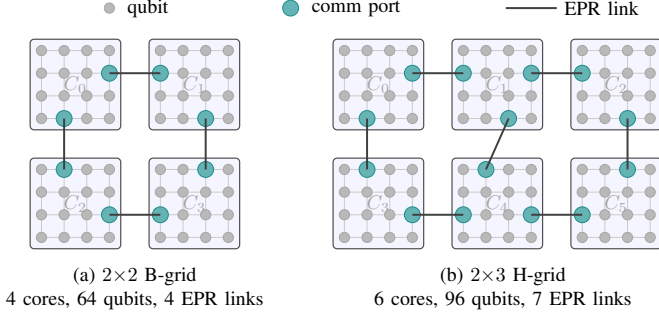


Fig. 2. Distributed architectures used in the experiments. Grey vertices are physical qubits, teal vertices are communication ports, and bold edges are inter-core EPR links. Each core is a 4×4 grid: (a) a 2×2 B-grid and (b) a 2×3 H-grid.

B. Teleportation

Quantum teleportation transfers a qubit state between two locations using a pre-shared Einstein–Podolsky–Rosen (EPR) pair and classical communication; the state is removed from the sender and reconstructed at the receiver [13]. In this paper, one consumed EPR pair is one unit of inter-core communication cost, and “EPR” as a unit (e.g. “10 EPRs”) denotes EPR-pair count.

Two communication primitives are relevant. *Teledata* moves a logical qubit to another core, after which subsequent gates may execute locally. *Telegate* implements a non-local 2Q gate without relocating either logical qubit [9]. Teledata can amortise its cost over a qubit’s subsequent residence in the destination core. Telegate can amortise one entangled state over a compatible group of remote gates, but this requires an available link qubit and entanglement that survives for the group duration; Section IV-E measures what those two assumptions are worth.

C. Distributed architecture

We model the distributed quantum computer as a weighted coupling graph $G_A = (V, E_A, w)$. Its physical-qubit vertices are partitioned into K cores, $V = V_1 \sqcup \dots \sqcup V_K$. Intra-core edges E_{intra} model local connectivity and have unit weight. Inter-core edges E_{inter} join designated communication ports and have weight $w_{\text{link}} = 10$, the cost of a teleport relative to a unit intra-core hop. A communication port is an ordinary physical slot that also terminates an inter-core link; it may hold a data qubit when it is not used for communication. We assume throughout that G_A is *connected*, and in the stronger form the router actually requires: the subgraph induced by each V_i is connected, and so is the core graph on $\{1, \dots, K\}$ induced by E_{inter} . Every distance below is then finite.

dsABRE uses three distance tables: physical distance d_{phys} , intra-core distance d_{intra} , and core-graph distance d_{core} . It therefore applies to arbitrary connected intra-core graphs and inter-core topologies. Our experiments instantiate the grid-of-grids architectures of TelesABRE [10], shown in Figure 2.

Physical distances need not be stored in a dense all-pairs table: every inter-core path exits and enters through ports, so

with Π_i the ports of core i , for $p \in V_i$ and $q \in V_j$,

$$d_{\text{phys}}(p, q) = \min_{\pi \in \Pi_i, \pi' \in \Pi_j} [d_{\text{intra}}(p, \pi) + d_{\text{port}}(\pi, \pi') + d_{\text{intra}}(\pi', q)]. \quad (1)$$

The implementation memoises these queries.

D. SABRE-style routing

SABRE [7] is a heuristic router that repeatedly executes ready gates and inserts a candidate SWAP when a front-layer 2Q gate is not adjacent. Its score favours a SWAP that reduces the distance of front-layer gates while also considering a bounded lookahead set. We use the LightsABRE form [14]:

$$H = \frac{1}{|F|} \sum_{g \in F} \Delta(g) + w_e \frac{1}{|E|} \sum_{g \in E} \Delta(g), \quad (2)$$

where F is the front layer, E is the extended set, w_e weights the lookahead term against the front-layer term, and $\Delta(g) = d_{\text{new}}^g - d_{\text{old}}^g$ is the distance change for gate g . A negative value denotes progress. dsABRE retains this front-layer-plus-lookahead principle, but adapts the candidate action and score to account for port staging and finite capacity in the destination core. Building on a heuristic that production toolchains already run bounds the compile time and keeps the output executable: the router carries the layout with it and admits only moves that layout allows. What remains open is what the SABRE objective must become when one chip becomes several.

III. dsABRE

dsABRE is a SABRE-style router for multi-core processors. It uses teledata alone: one teleport consumes one EPR pair and moves one logical qubit across one adjacent-core link. Thus, unlike an amortised telegate, no entanglement lifetime or live EPR buffer is modelled.

The routing state is a layout ℓ and inverse ℓ^{-1} , a DAG W of unexecuted gates, its front layer F of gates with no unexecuted predecessors, and a bounded extended set E beyond F . A front-layer 2Q gate is *intra-core* when both operands are in one core and *inter-core* otherwise. Table I lists the notation.

A. Routing Policy, Score, and Legality

dsABRE has two actions: an intra-core SWAP and an inter-core teleport. It first resolves non-adjacent intra-core front gates by local SWAPs. Only when none remain does it score teleports for inter-core front gates. This priority rule avoids imposing an artificial exchange rate between the local and global scorers.

A teleport may require port eviction and staging SWAPs, and consumes one free slot in its landing core. We treat these operations and the teleport as one *macro-action*: a SWAP is a single primitive, while a teleport is a short sequence scored and applied as one unit. *Atomic* below means that such a sequence either completes or leaves the state untouched. Recovery (Section III-F) has the same property, but is the fallback for when neither ordinary action makes progress rather than a third ordinary action. If $\Delta\Phi(a)$ is the change action a induces in a front-layer-plus-lookahead routing potential—negative when a

TABLE I
NOTATION FOR SECTION III. DEFAULTS ARE FIXED ACROSS ALL
EXPERIMENTS AND ARCHITECTURES.

Symbol	Meaning	Default
ℓ, ℓ^{-1}	layout and inverse layout	—
W, F, E	remaining DAG, front layer, extended set	—
F_c, E_c	restrictions of F, E to core c	—
K, M, P, n	cores, qubits/core, physical, logical qubits	—
$d_{\text{intra}}, d_{\text{core}}$	intra-core and core-graph distance	—
D_M, D_K	intra-core and core-graph diameter	—
$\text{dep}(g)$	BFS depth of g in E	—
π_s, π_d, n_s	source and destination ports; staging slot	—
$d_{\text{prep}}, c_{\text{cap}}$	preparation cost, capacity penalty	—
Δ_F, Δ_E	front-layer and lookahead score change	—
f_c, F_{free}	free slots in core c ; total free slots	—
$c_{\text{swap}}, c_{\text{tele}}$	local-SWAP and teleport cost	3, 10
w_e	lookahead weight	0.25
L, γ	extended-set capacity and depth decay	20, 0.9
τ, c_{pen}	capacity threshold and penalty	3, 15
ϕ_1	legality floor (Eq. 4)	2
L_{deadlock}	stalled-iteration limit	$4D_K(D_M + 1)$

leaves the waiting gates closer to executable—the conceptual objective is

$$s(a) = \underbrace{\text{cost}(a)}_{\text{resource spent}} + \underbrace{\text{pen}(a)}_{\text{capacity consumed}} + \underbrace{\Delta\Phi(a)}_{\text{progress bought}}. \quad (3)$$

All compared teleport candidates consume one EPR pair, so this constant is omitted from their ranking.

Legality rule: The capacity penalty ranks feasible moves; it does not determine feasibility. Feasibility is enforced by a separate *legality floor*: an ordinary teleport is generated only if its landing core c_{next} has

$$f_{c_{\text{next}}} \geq \phi_1 = 2, \quad (4)$$

the free slots being counted before the move. A teleport consumes one slot of its landing core, so after the arrival that core still retains at least $\phi_1 - 1 = 1$ free slot, and local SWAPs preserve every core’s occupancy. Hence ordinary routing never creates a full core, whose occupied communication port could otherwise prevent a later departure. Equation 4 is used in candidate generation and in the recovery proof.

B. Workflow

Figure 3 summarises the routing loop and Algorithm 1 states it in full. The router retains a *progress checkpoint*—the layout and its inverse, the remaining DAG W , the *trace* of SWAPs, teleports and executed gates emitted so far, and the counters that summarise them—as they stood immediately after the last decrease in $|W|$, and initially the input layout, the full DAG and an empty trace. Restoring it rewinds the emitted circuit as well as the layout, so what is emitted always replays against the state it is restored to. Each iteration is:

P1: Execute. Repeatedly execute front-layer 1Q gates and adjacent intra-core 2Q gates. Stop if W is empty.

P2: Classify. Partition the remaining front layer into non-adjacent intra-core gates F_{intra} and inter-core gates F_{inter} .

P3: Route. If $F_{\text{intra}} \neq \emptyset$, apply the lowest-score local SWAP. Otherwise enumerate legal teleport macro-actions for

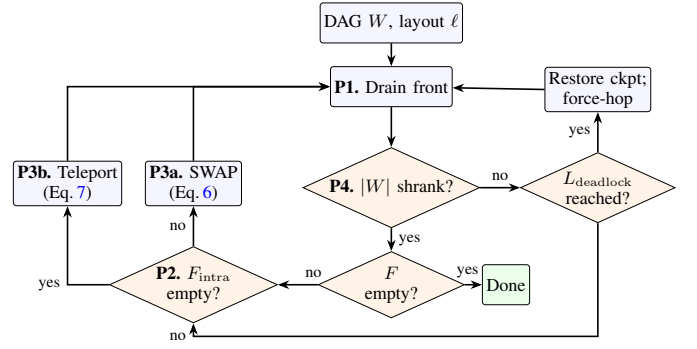


Fig. 3. dSABRE routing workflow. P1 drains executable gates; P2 classifies the remaining front layer; P3 applies either the best local SWAP or the best legal teleport macro-action; and P4 checkpoints every decrease in the remaining DAG. Three conditions invoke the guaranteed transaction of Section III-F: an empty legal candidate set, exhausted candidate attempts, and L_{deadlock} iterations without progress. Only the third is drawn; the two edges from P3b to the force-hop node are omitted for clarity, and Lemma 2 shows the second never fires.

F_{inter} and try them in score order. Each attempt is atomic: a failed eviction, staging sequence, or teleport restores the pre-attempt state. If no candidate is legal or none commits, restore the progress checkpoint and invoke recovery; by Lemma 2 the second case cannot arise.

P4: Checkpoint or recover. A decrease in $|W|$ replaces the checkpoint and resets the stalled counter. Otherwise increment it; after L_{deadlock} stalled iterations, restore the checkpoint and invoke recovery. Recovery is atomic: it either retires its target gate and restores the reserve, or restores its entry state—a branch Theorem 1 rules out, leaving it an assertion on the hypotheses rather than an expected outcome.

C. Intra-Core SWAP Heuristic

For active core c , let F_c be its non-adjacent intra-core front gates and let $E_c \subseteq E$ contain the extended-set gates whose operands currently both reside in c . The global construction of E supplies one consistent depth $\text{dep}(g)$ for both scorers. For a candidate SWAP, define

$$\tilde{\Delta}_E^c = \sum_{g \in E_c} \gamma^{\text{dep}(g)} (d_{\text{new}}^g - d_{\text{old}}^g), \quad (5)$$

where $0 < \gamma \leq 1$ and both distances are d_{intra} . The candidates are the physical edges (u, v) within c incident to an operand of a gate in F_c ; the selected SWAP minimises

$$H_{\text{intra}} = \frac{\Delta_F^c}{|F_c|} + w_e \frac{\tilde{\Delta}_E^c}{|E_c|}. \quad (6)$$

When $E_c = \emptyset$ the lookahead term is zero. This is the SABRE objective of Eq. 2 with one change: SABRE weights every gate of its extended set equally, having filled that set in topological order, which fixes no notion of how far ahead a gate lies. Here $\text{dep}(g)$ is the layer of g in a *breadth-first* (BFS) expansion of the remaining DAG—one dependency beyond the front layer is $\text{dep} = 1$, one beyond that $\text{dep} = 2$ (Section III-E)—so $\gamma^{\text{dep}(g)}$ discounts by exactly that. Candidate evaluations are independent across active cores.

Algorithm 1 dsABRE route(C, ℓ_0). An empty legal candidate set goes straight to recovery rather than waiting out the stall window, since no local SWAP can create headroom in a neighbouring core. RECOVER is the transaction of Section III-F.

```

1:  $W \leftarrow$  working DAG of  $C$ ;  $\ell \leftarrow \ell_0$ ; initialise  $\ell^{-1}$ 
2: verify the preconditions of Theorem 1; initialise a check-
   point
3: while  $W \neq \emptyset$  do
4:   Drain executable 1Q gates and adjacent intra-core front-
     layer 2Q gates
5:   if  $W = \emptyset$  then
6:     break
7:   end if
8:   Form  $F$ , partition it into  $F_{\text{intra}}$  and  $F_{\text{inter}}$ , and build  $E$ 
     if stale
9:   if  $F_{\text{intra}} \neq \emptyset$  then
10:    for each active core  $c$  with  $F_c \neq \emptyset$  do
11:      Apply the minimum-score intra-core SWAP of
        core  $c$  using Eq. 6
12:    end for
13:  else
14:    Enumerate teleport macro-actions for  $F_{\text{inter}}$  whose
      landing core  $c'$  has  $f_{c'} \geq \phi_1$ 
15:    Try candidates in score order; each attempt either
      commits atomically or restores its entry state
16:    if the candidate set is empty, or no candidate commits
      then
17:      restore the checkpoint and invoke RECOVER( $W, \ell$ )
18:    end if
19:  end if
20:  if  $|W|$  decreased then
21:    replace the checkpoint and reset the stalled counter
22:  else if the stalled counter reaches  $L_{\text{deadlock}}$  then
23:    restore the checkpoint and invoke RECOVER( $W, \ell$ )
24:  end if
25: end while
26: return routed circuit, final layout, and metrics

```

D. Inter-Core Teleportation Scoring

For an inter-core front gate $g = (q_1, q_2)$, both operands are considered as sources. For source q_1 in c_{src} , the router enumerates each adjacent core c_{next} and each link (π_s, π_d) joining them, retaining only candidates that satisfy Eq. 4. Let $p_1 = \ell(q_1)$ and let n_s be a neighbour of π_s minimising $d_{\text{intra}}(p_1, n_s)$. The macro-action clears both ports, stages q_1 to n_s , and teleports it across (π_s, π_d) . Each port has a free slot to be cleared into: the source core has one because ordinary routing leaves no core full, as Section III-F proves, and Eq. 4 gives the landing core two, of which one absorbs the port's occupant and the arrival still leaves one behind.

The score is

$$s = d_{\text{prep}} + c_{\text{cap}} + \Delta_F + w_e \tilde{\Delta}_E, \quad (7)$$

where lower is better and

$$d_{\text{prep}} = d_{\text{intra}}(p_1, n_s) + d_{\text{evict}}(\pi_s) + d_{\text{evict}}(\pi_d), \quad (8)$$

$$c_{\text{cap}} = c_{\text{pen}} \max\{0, \tau - f_{c_{\text{next}}}\}, \quad (9)$$

$$\Delta_F = d_{\text{new}}^g - d_{\text{old}}^g, \quad (10)$$

$$\tilde{\Delta}_E = \sum_{h \in E(q_1)} \gamma^{\text{dep}(h)} (d_{\text{new}}^h - d_{\text{old}}^h). \quad (11)$$

Here $d_{\text{evict}}(\pi)$ is the number of SWAPs that clear port π , and is zero when π is empty. $E(q_1) \subseteq E$ is the set of extended-set gates having q_1 as an operand, and for a gate h with operands q_1 and q' , $d_{\text{old}}^h = d_{\text{phys}}(p_1, \ell(q'))$ and $d_{\text{new}}^h = d_{\text{phys}}(\pi_d, \ell(q'))$: the teleported qubit is priced at the destination port it lands on. Δ_F is measured with the same d_{phys} , so the two progress terms are on one scale; and unlike Eq. 6, neither is divided by a set size, because a teleport moves one operand of one gate and front-layer gates are qubit-disjoint, so exactly one front-layer term ever changes. The candidates generated for every gate of F_{inter} are pooled into a single ranking, so one arg min chooses which remote gate to advance as well as how to advance it. The preparation term prices availability: an occupied port is costly, not silently excluded. The capacity term prices the immediate landing core, not the core holding the other operand. All distance terms use the weighted architecture graph G_A —an intra-core edge costs one and an inter-core link 10—so d_{prep} , Δ_F , and $\tilde{\Delta}_E$ share a cost scale. Equation 11 is an approximation adopted for efficiency in two respects: it sums only over gates involving the teleported qubit, ignoring extended-set gates the move affects through its evictions, and it scores the layout after the teleport rather than the intermediate staging and eviction layouts.

Running example: Figure 4 and Table II score the three outgoing-link candidates on the 2×3 H-grid at defaults, isolating one factor each, and show the three groups of Eq. 3 deciding between them: A wins on the cheapest staging plus the largest progress; B moves away from C_5 , turning Δ_F against it; C buys A's progress but lands in a nearly-full core, where the capacity term alone reverses the decision.

E. Depth-Ordered Extended Set

This section constructs the inter-core extended set E of Eq. 11. The set is a breadth-first expansion of the remaining DAG. Gates in BFS layer k receive $\text{dep}(g) = k$; within a layer, gates sharing a qubit with a front-layer gate are emitted first. Construction stops after L gates, so every retained gate is no deeper than any omitted gate. Maintained remaining in-degrees give $O(L)$ construction time per update.

The intra-core sets E_c of Eq. 6 are breadth-first for the same reason: they are read off this one set as the gates of E whose two operands currently reside in c , carrying the depths E already assigned, so one construction serves both scorers and $\text{dep}(g)$ means the same thing in each. Section IV-D measures what the BFS order is worth against a topological one.

F. Capacity Conditions and Recovery

The ordinary score is a preference rule, not a proof that an action will always exist. Recovery handles the exceptional

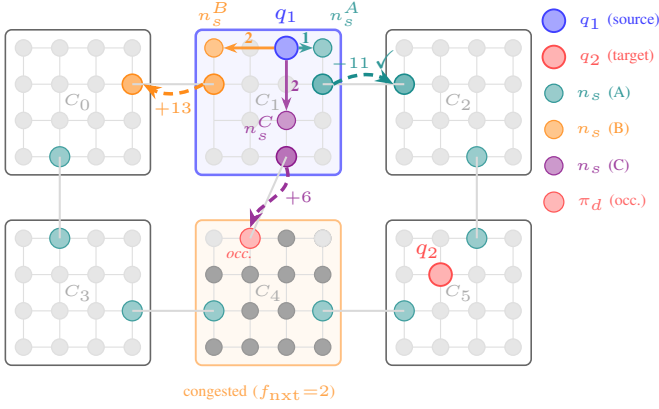


Fig. 4. How dSABRE chooses where to teleport a qubit. A single cross-core gate $g=(q_1, q_2)$ is pending, with q_1 in core C_1 (blue border) and its partner q_2 in C_5 ; the router scores the three links leaving C_1 and takes the cheapest by Eq. 7. Executing any one of them takes two steps: intra-core SWAPs first move q_1 to the staging slot n_s (count on the solid arrow), then a teleport along the link (π_s, π_d) carries it into the next core (dashed arc, annotated with the resulting score s). Candidates A (teal), B (orange) and C (violet) correspond row by row to Table II; C_4 's orange border marks that only two of its slots are free, and the red port marked *occ.* is already occupied.

TABLE II

SCORE BREAKDOWN FOR THE THREE CANDIDATES OF THE RUNNING EXAMPLE (FIGURE 4). LOWER IS BETTER; CANDIDATE A IS SELECTED.

Candidate	next core	direction	d_{prep}	c_{cap}	Δ_F	s
A	C_2	towards C_5	1	0	-12	-11
B	C_0	away from C_5	2	0	+11	13
C	C_4	towards C_5	3	15	-12	6

state in which ordinary routing stalls, by a *guaranteed transaction* that provably retires the blocked gate.

A free slot is a physical site holding no logical qubit; port eviction moves a port's occupant to a free site in the same core, so any non-full core can clear an occupied port.

Let f_c be the number of free slots in core c and let $F_{\text{free}} = \sum_c f_c = P - n$ be the total number of free slots. Recovery keeps a *reserve* of two free slots per core—one to evacuate a port with, one to receive an arriving state—so before routing we require

$$F_{\text{free}} \geq 2K + 1, \quad (12)$$

and initially

$$f_c \geq 2 \quad \text{for every core } c. \quad (13)$$

The arguments below need only that the reserve is at least two; a larger one costs a proportionally larger Eq. 12. The total number of free slots is conserved. The reserve is required only on entry to and exit from recovery; ordinary routing need not preserve it, and measurements in the online appendices show that on the 64-qubit suite some core sits below it at 24.6% of action boundaries.

Two consequences of Eq. 4 are used repeatedly and are recorded here.

Lemma 1 (Ordinary routing creates no full core). *Assume Eq. 13 holds of the initial layout. Then after every ordinary action—and after every rollback—every core has at least one free slot.*

Proof. Induct over the states the router reaches. Local SWAPs, port evictions among them, leave every per-core count unchanged. An ordinary teleport decreases only its landing core's count, which Eq. 4 puts at two or more before the move, and increases its source's. A rollback restores a state the router has already reached, for which the claim holds by the induction hypothesis; it is therefore the only step that needs the checkpoint to be a past state rather than an arbitrary one. Only $f_c \geq 1$ is used; Eq. 13 supplies it with a slot to spare. $\square$

Lemma 2 (Legal candidates commit). *Assume Lemma 1, that every core's coupling graph is connected, and that the staging slot n_s is chosen among the neighbours of π_s lying in the source qubit's own component of that graph with π_s deleted. Then every teleport macro-action admitted by Eq. 4 completes: no candidate that is generated is rejected at execution time.*

Proof. A macro-action fails only if it cannot evacuate its source or destination port, cannot stage its qubit, or cannot land the arriving state. By Lemma 1 both ports lie in cores holding a free slot, and intra-core connectivity moves that vacancy along a path to the port, so both evictions succeed. The staging SWAPs must then avoid π_s ; a path through it would push a bystander onto the port on its last SWAP and undo the eviction. Deleting π_s can disconnect the core—on the heavy-hex tiles used in Section IV-C both ports are cut vertices, though on grid cores none is—so the nearest port neighbour is not always reachable. Some neighbour always is: every component of the core minus π_s is adjacent to π_s , because the core is connected, so the source qubit's own component contains one, and the lemma's hypothesis selects it. Staging SWAPs preserve occupancy. The arrival then consumes one of the at least two free slots Eq. 4 requires of the landing core. $\square$

Recovery transaction: Recovery uses two distinct cores, and they are chosen for different reasons. A *donor* is any core with a free slot to spare, $f_{cd} \geq 3$; Eq. 12 guarantees one exists, since otherwise every core would hold at most two and $F_{\text{free}} \leq 2K$. The *meeting core* is where the blocked gate will execute. They coincide only by accident. A *relay* advances one vacancy one hop along a shortest core-graph path from a donor towards a core in deficit, by teleporting a *filler*—any qubit other than the two operands of the blocked gate, which the transaction holds fixed—the opposite way.

Algorithm 2 states the transaction; its steps are sequential, and the teleports it emits are not themselves subject to Eq. 4, the reserve playing that role for them. Line 6 admits two families of meeting core. If $m \in \{a, b\}$ only one operand moves, and line 7 tops m up to three free slots so that it is back at the reserve once the arrival lands. If m is a third core both operands arrive, so m is proposed only when it *already* holds four: no relay is requested, and none could be promised, since Eq. 12 guarantees one surplus vacancy and not two. The proof below uses only the first family; third-core rendezvous shortens the operand walks when a core happens to have the local slack, and completion never depends on it.

Theorem 1 (Termination with success). *Suppose the core graph and every intra-core coupling graph are connected,*

Algorithm 2 RECOVER(W, ℓ): the guaranteed transaction of Section III-F, which retires one blocked inter-core gate and returns with Eq. 13 restored.

-
- 1: record the entry state; on any failed precondition below, restore it and abort
 - 2: pick a blocked inter-core front gate $g = (q_1, q_2)$, in cores a and b
 - 3: **while** some core c_t has $f_{c_t} < 2$ **do**
 - 4: relay a vacancy to c_t from a donor {re-establish Eq. 13}
 - 5: **end while**
 - 6: choose m minimising $d_{\text{core}}(a, m) + d_{\text{core}}(b, m)$ plus its relay shortfall, over $m \in \{a, b\}$ and over third cores already holding $f_m \geq 4$
 - 7: relay vacancies until $f_m \geq 3$ {no-op for a third core}
 - 8: teleport each operand not in m to m , hop by hop along a shortest path
 - 9: SWAP within m until $\ell(q_1), \ell(q_2)$ are adjacent; execute g
 - 10: **return** the updated W , with g retired, and ℓ
-

every core has at least five physical qubits, Eqs. 12–13 hold initially, the staging rule of Lemma 2 is used, and checkpoint–rollback is enabled. Then dSABRE routes every gate of the input DAG and does not abort.

Proof. F_{free} is conserved, so Eq. 12 supplies a donor in every reachable state, and by Lemma 1 no ordinary action creates a full core. It suffices to show that one invocation of Algorithm 2 always retires its gate and returns in a state again satisfying Eq. 13.

Relays (line 3). Let c_t have $f_{c_t} < 2$ and take a shortest core path $c_0, \dots, c_h = c_t$ from a donor c_0 . Hop i teleports a filler from c_i to c_{i-1} , so c_{i-1} loses a slot and c_i gains one; performing the hops in the order $i = 1, \dots, h$ keeps every core’s count at or above its entry value except c_0 , which ends one down but started at three. Each hop is an ordinary teleport primitive and is legal for the reasons of Lemma 2, its receiving core holding a free slot throughout—by Lemma 1 for c_0 , by the preceding hop for the rest. The filler exists because c_i must hold some qubit other than the two operands the transaction is holding fixed; a core holding nothing else has $M - 2 \geq 3$ free slots, so it is a donor and the walk starts there instead—this is where, and only where, the five-site hypothesis is used. Each repetition reduces the total deficit $\sum_c \max\{0, 2 - f_c\}$ by one, so finitely many restore Eq. 13.

Rendezvous (lines 6–7). Take $m = a$. Every core now holds two free slots and $F_{\text{free}} \geq 2K + 1$, so some core holds three; one relay from it, by the argument above, leaves $f_a \geq 3$ without pushing any core below two. Walk the operand in b to a along a shortest core path. An intermediate core dips to one free slot while it holds the operand—enough to clear its outgoing port—and returns to two when the operand leaves; b ends one up and a one down, at $f_a \geq 2$. So Eq. 13 holds when the walk ends, with both operands in a : this is why line 7 tops a up first rather than repairing afterwards. Intra-core connectivity then makes the operands adjacent by SWAPs, which change no count, and the gate executes.

So the transaction strictly decreases $|W|$ and preserves Eq. 13. Between two decreases of $|W|$, the ordinary loop performs at most L_{deadlock} stalled iterations before recovery is invoked. Strong induction on $|W|$ proves that the router

reaches $|W| = 0$ in finitely many steps and never aborts. $\square$

Satisfying the hypotheses: The hypotheses are mild, and every device and workload evaluated in this paper satisfies them with substantial margin. Connectivity is needed to route at all: a core whose coupling graph is disconnected cannot bring two of its own qubits together, and a disconnected core graph can only execute circuits independently within its connected components. The size condition asks a core to hold five physical qubits, the smallest used here holding 16. Checkpoint–rollback is an implementation property, enabled by default. It is not what re-establishes Lemma 1, which holds throughout ordinary routing anyway: its role is to discard a branch that has stopped making progress and to leave the emitted trace and the restored layout consistent with each other before the deterministic transaction begins. Equation 13 is a property of the initial layout rather than something the user must arrange: the procedure of Section III-G reserves at least two sites in every core and so establishes it by construction. That leaves Eq. 12 as the only condition constraining the workload, and it constrains it loosely; the device may be filled to $1 - (2K + 1)/P$, which is 86% for the 16-site cores used here and 91–92% for the 25-site ones. The tightest instance in this paper, the 64-qubit suite on the H-grid, has 32 free slots where 13 suffice. A circuit that violates Eq. 12 is rejected before routing rather than part-way through.

The heavy-hex star of Section IV-C is the tightest evaluated case and clears every hypothesis. Its hub is a cut vertex of the core graph, so every relay between two leaves crosses it—longer walks, but no step made illegal—and two of the hub tile’s three ports are cut vertices of its own coupling graph, which is the situation Lemma 2 handles and which no grid core presents. Capacity is not close: four 27-site cores hold 64 logical qubits, leaving 44 free slots where nine suffice.

G. Initial Layout

The default layout bootstraps from Qiskit’s SabreLayout on the full architecture graph, but reserves slots in every core. It withholds the k minimum-degree sites that are farthest, by total shortest-path distance, from the rest of their core—corners on grids and leaves on heavy-hex cores. The value $k \geq 2$ is chosen so that the resulting layout satisfies Eq. 13; the reservation is per core rather than a whole-device quota. Forward, backward, and final-forward passes are run, retaining the better forward route by EPR count. Three SabreLayout seeds are used, so one compilation performs nine routes in all, of which the reported one is the best forward pass.

H. Complexity

Let N be the number of gates in the circuit. Distance and shortest-path tables are precomputed once per architecture and are not charged below. Here L , the intra-core degree, the core-graph degree, and the number of links joining one adjacent pair of cores are constants, so a core has $O(M)$ edges, the core graph has $O(K)$, and there are $O(K)$ communication ports. The last two are what make the teleport candidate count $O(n)$ below. All-pairs breadth-first search inside every core then

costs $O(KM^2)$ for d_{intra} , and $O(KM^2D_M)$ to materialise the intra-core paths, whose KM^2 entries have $O(D_M)$ sites each; the same search on the core graph costs $O(K^2)$ for d_{core} and $O(K^2D_K)$ for the core paths. Dijkstra over the port graph gives the port-level table d_{port} in $O(K^2 \log K)$, and d_{phys} is then composed on demand by Eq. 1 rather than materialised, which avoids the $\Theta(P^2)$ table a whole-chip all-pairs search would build. The precomputation is therefore

$$T_{\text{pre}} = O(KM^2D_M + K^2(D_K + \log K)), \quad (14)$$

in $O(KM^2 + K^2)$ space.

The extended set is rebuilt only after a gate retires, at the $O(L)$ per update of Section III-E and hence $O(NL) = O(N)$ over a route. In one local-routing iteration, each active core enumerates $O(M)$ candidate edges—its front gates are qubit-disjoint, so $|F_c| \leq M/2$ —and scores each in $O(L)$, since a SWAP moves at most two front-layer operands; over all active cores the iteration costs $O(n)$, because $\sum_c |F_c| \leq n/2$. One teleport-selection iteration costs $O(M(K+n))$: $O(KM)$ for the free-slot census and $O(nM)$ for the $O(n)$ candidates, each of which scans its landing core for an eviction target. That last scan is an implementation bound, not an inherent one: maintaining per-core free-site and port-occupancy structures would replace it by a lookup. The teleport iteration therefore dominates.

Since recovery follows after at most $L_{\text{deadlock}} = 4D_K(D_M + 1)$ stalled iterations and retires at least one gate, the number of ordinary iterations is $O(ND_K(D_M + 1))$, so

$$T_{\text{ord}} = O(ND_K(D_M + 1)M(K+n)). \quad (15)$$

One recovery transaction costs $O(KM(K + D_K))$ time and occurs at most once per retired gate: it relays $O(K)$ vacancies over core paths of at most D_K hops, each hop scanning its core in $O(M)$ for a filler and a free slot, over an $O(KM)$ donor search, giving $T_{\text{rec}} = O(NKM(K + D_K))$. Summing the extended set, T_{ord} and T_{rec} , and using $D_M, D_K, M \geq 1$ and $K = O(n)$, and absorbing $KD_K \leq K^2$,

$$T_{\text{total}} = O(NM(D_KD_Mn + K^2)). \quad (16)$$

This is a deliberately loose common bound over both regimes; the K^2 term is the recovery contribution and is charged once per gate even though Section IV-D measures recovery firing on a small fraction of them. For grid-of-grids architectures, $D_K = O(\sqrt{K})$ and $D_M = O(\sqrt{M})$, so $D_KD_M = O(\sqrt{P})$ with $P = KM$, giving $T_{\text{total}} = O(NM(\sqrt{P}n + K^2))$.

IV. EXPERIMENTAL EVALUATION

A. Setup

Goal. We evaluate whether dSABRE reduces EPR consumption while retaining practical compile time and completing instances on finite-capacity multi-core devices. Our primary comparison is with TelesABRE [10], because it uses the same physical architectures, counts local SWAPs, and supports one-EPR teledata moves. It is also the strongest baseline of its family: it reports a 28% geometric-mean reduction in inter-core state transfers over Hungarian Qubit Assignment [4], [15], the partitioning-based multi-core mapper it measures

itself against. The comparison with `pytket-dqc` is reported separately: its EPR unit is the same, but an EPR pair there represents different work and assumes a different machine model.

Benchmarks and architectures. We use 21 MQT Benchmark circuits [16], transpiled to the IBM native basis with Qiskit optimisation level 3; measurements and barriers are removed. The 25-qubit suite contains AE, QFT, QNN, Random, GHZ, and Graphstate. The 36-qubit suite contains BV, DJ, QAOA, QPEexact, VQE-SU2, and W-state. The 64-qubit suite contains AE, QFT, QNN, Random, GHZ, Graphstate, QPEexact, QAOA, and a 16-bit Multiplier. The scalability study uses two families, QFT and QPEexact, at 100, 200, and 360 logical qubits.

The 25- and 36-qubit suites run on a 2×2 B-grid with four 4×4 cores (64 physical qubits and four inter-core links). The 64-qubit suite runs on a 2×3 H-grid with six 4×4 cores (96 physical qubits and seven links). These are the TelesABRE benchmark topologies. We additionally use four 27-qubit heavy-hex cores connected as a ring or star, and 5×5 grid cores for the scalability experiments. Every core graph and every intra-core coupling graph is connected, as Section II-C assumes. Counting a circuit once per architecture on which it is routed—the 21 suite circuits, the six scalability instances, and the nine 64-qubit circuits on each of the two heavy-hex networks—the evaluation covers 45 instances.

Protocol. Unless stated otherwise, dSABRE uses the fixed parameters in Table I and its default capacity-reserving initial layout. dSABRE and TelesABRE each report the best result from three seeds, following their standard use; `pytket-dqc` reports the best of five. We report EPR pairs and local SWAPs. For TelesABRE, EPR is the sum of teledata and telegate operations. A tool is reported as non-convergent on an instance when none of its seeds returns a valid routing. For TelesABRE this is its own verdict rather than a limit we impose: every seed exhausts its ten internal attempts at the tool's own deadlock criterion, well inside the wall-clock budget of 300 s per seed (900 s on the scalability series). A failed run is reported explicitly and is excluded from a baseline-only geometric mean; the corresponding dSABRE mean over all completed circuits is also shown. Per-seed results and commands are provided in the supplementary material.

B. Main Comparison with TelesABRE

Table III gives the main results. dSABRE lowers geometric-mean EPR consumption by 50.8%, 62.0%, and 49.1% on the 25-, 36-, and 64-qubit suites, respectively, on the circuits completed by both routers. It also uses fewer geometric-mean local SWAPs in every suite. At 64 qubits, TelesABRE fails on Random in all three attempts, whereas dSABRE completes it using 732 EPR pairs. The result is therefore not only an average EPR reduction: dSABRE also completes one additional circuit under the same architecture and run budget. Over the whole evaluation, dSABRE completes 45 of the 45 circuit-architecture instances and TelesABRE 40 of 45, under the run budget of Section IV-A.

TABLE III
MAIN COMPARISON WITH TELESABRE. EPR IS TELEDATA PLUS TELEGATE
FOR TELESABRE AND TELEDATA FOR DSABRE. NEGATIVE Δ FAVOURS
DSABRE. *TELESABRE FAILS TO CONVERGE.

Circuit	CX	TelesABRE		dsABRE		Δ EPR (%)
		EPR	SWAP	EPR	SWAP	
<i>25 qubits, 2×2 B-grid</i>						
AE	558	26	292	23	288	-11.5
GHZ	24	4	36	1	4	-75.0
Graphstate	25	13	47	2	15	-84.6
QFT	580	46	405	33	332	-28.3
QNN	1223	54	659	47	661	-13.0
Random	1124	271	1248	181	1208	-33.2
Gmean (6)		31.1	234	15.3	129	-50.8
<i>36 qubits, 2×2 B-grid</i>						
BV	17	6	39	1	13	-83.3
DJ	35	4	63	3	23	-25.0
QAOA	1200	268	1165	134	1017	-50.0
QPEexact	1019	190	883	62	612	-67.4
VQE-SU2	105	28	134	9	45	-67.9
W-state	70	13	79	6	40	-53.8
Gmean (6)		27.6	173	10.5	83	-62.0
<i>64 qubits, 2×3 H-grid</i>						
AE	1962	353	1494	165	1325	-53.3
GHZ	63	16	132	6	39	-62.5
Graphstate	64	62	281	15	88	-75.8
QFT	1966	312	1711	180	1480	-42.3
QNN	8126	889	5471	539	5260	-39.4
Random*	1627	—	—	732	3603	—
QPEexact	2139	289	1706	187	1576	-35.3
QAOA	3920	956	4189	555	3734	-41.9
Multiplier	13040	1601	9654	1285	8969	-19.7
Gmean (8 common)		284.5	1564	144.7	1081	-49.1
Gmean (9 dsABRE)		—	—	173.3	1236	—
<i>Scalability, 5×5 cores throughout, core count 6 → 12 → 20</i>						
QFT 100q	3420	248	2414	201	2452	-19.0
QFT 200q	7220	1232	7089	766	6482	-37.8
QFT 360q*	13300	—	—	1890	15606	—
QPEexact 100q	3579	586	3485	246	2836	-58.0
QPEexact 200q*	7579	—	—	896	6834	—
QPEexact 360q*	13979	—	—	1938	16090	—

C. Robustness, Scale, and Runtime

Topology. The core here is a 27-qubit IBM heavy-hex tile, which shares none of the grid’s regularity—degrees over $\{1, 2, 3\}$, six degree-1 leaves, and diameter 12 against the 4×4 grid’s 6—and four such tiles are wired as a ring (4 links) and as a star (3 links, with a hub every inter-core route must cross). Nothing is specialised for either: the corner reservation of Section III-G reads vertex degree and remoteness, so it selects leaf qubits where it selected grid corners. On the ring both routers complete all nine 64-qubit circuits and dsABRE reduces EPRs by 52.0% against TelesABRE; on the star the reduction is 41.5%, with TelesABRE again failing on Random where dsABRE uses 475 EPR pairs. The improvement is therefore not specific to regular grid cores or a mesh core graph.

Scale. We scale two circuit families, QFT and QPEexact, while holding core occupancy approximately constant: 100, 200, and 360 logical qubits use 6, 12, and 20 5×5 cores, respectively. On QFT, dsABRE uses 201 EPR pairs at 100 qubits against TelesABRE’s 248, and 766 against 1232 at 200 qubits; at 360 qubits TelesABRE does not converge in any of three runs, while dsABRE completes at 1890. On QPEexact

the margin is larger and the baseline gives out earlier: 246 EPR pairs against 586 at 100 qubits, and no convergence at either 200 or 360 qubits, where dsABRE completes at 896 and 1938. Three of the five non-convergent instances in this paper are therefore these large routes; the other two are Random at 64 qubits, on the H-grid and on the heavy-hex star. These cases show that dsABRE’s capacity-aware routing and recovery framework remains operational as the core graph grows. They are not evidence about *why* the baseline stops: non-convergence there could equally reflect search behaviour, candidate cycling, or implementation limits, and we do not attribute it to capacity handling.

Runtime. On the eight 64-qubit circuits completed by TelesABRE, the geometric-mean compilation time is 1.9 s for dsABRE, 4.7 s for TelesABRE, and 107 s for `pytket-dqc`; on the largest circuit, the 13 040-CX Multiplier, the three take 23.7, 136.3, and 1393 s. The gap is not uniform—on QNN at 8126 CX the two routers are within 0.2 s—and the tools use different search procedures, so these are end-to-end practical costs rather than an operation-level comparison. Timing includes the complete routing or distribution attempt; for dsABRE that is layout generation and all nine routes. Within one architecture, compile time tracks gate count: a log-log fit of per-seed time against CX count gives an exponent of 0.97 ($R^2=0.95$) over the nine 64-qubit circuits, and 0.92 and 1.21 over the 36- and 25-qubit suites. Over the six scalability instances the fitted exponent is 2.2 instead, because there the device grows with the circuit—twenty 5×5 cores at 360 qubits against six at 100—which Eq. 16 charges through its K^2 term. The near-linear reading is therefore a statement about circuit size at fixed hardware, not about scaling the machine.

D. Ablation and Completion

Table IV isolates the main design choices on the complete 64-qubit suite. Replacing the BFS depth-ordered extended set with a topological-order set increases geometric-mean EPR by 12.4%. Removing the capacity score increases it by 9.4%. Most importantly, removing both the capacity score and the legality floor of Eq. 4 aborts QNN and Multiplier; over the seven circuits that all configurations complete, it costs 87.8% more EPR pairs. This supports two distinct claims: the score improves EPR efficiency, while the floor supplies the completion condition.

Table V counts how often the guarantee is invoked. A *transaction* is one invocation of Algorithm 2 and retires exactly one gate; a *relay* is one hop of its vacancy walk. The guarantee is rarely used on the reported small and medium instances: no selected route among the 21 circuits at 25–64 qubits invokes recovery. It becomes relevant at scale, where recovery retires 6, 10, and 66 gates over the reported 100-, 200-, and 360-qubit routes. Across all 1074 transactions no asserted recovery precondition failed, and none of the 1.08 million candidates scored was rejected at execution time, as Lemma 2 guarantees. Hence the guarantee is a fallback rather than the source of the main-suite EPR gains.

A shared-layout control further separates layout from routing. Starting dsABRE from TelesABRE’s initial layout in-

TABLE IV

ABLATION ON THE 64-QUBIT SUITE. ALL SETTINGS OTHER THAN THE NAMED COMPONENT ARE UNCHANGED. EVERY Δ COMPARES AGAINST FULL DSABRE ON THE SAME CIRCUITS: THE LOWER BLOCK IS EVALUATED ON A SUBSET, SO ITS *ref.* COLUMN REPEATS FULL DSABRE’S MEAN OVER THAT SUBSET AND ITS VALUES MUST NOT BE READ AGAINST THE NINE-CIRCUIT BLOCK ABOVE.

Configuration	circ.	gmEPR	ref.	Δ (%)	aborts
<i>All nine circuits</i>					
Full dsABRE	9	173.3	—	—	0
Topological set, not BFS	9	194.8	173.3	+12.4	0
No capacity score ($c_{\text{pen}}=0$)	9	189.5	173.3	+9.4	0
No legality floor (Eq. 4)	9	173.6	173.3	+0.2	0
Strict floor ($\phi_1=3$)	9	201.5	173.3	+16.3	0
<i>Common-subset controls</i>					
Neither floor nor score	7	207.9	110.7	+87.8	2
TelesABRE layout, not ours	8	189.4	144.7	+30.9	0

The “neither” row is compared on the seven circuits it completes; the TelesABRE-layout row on the eight for which a TelesABRE layout exists.

TABLE V

GATES RETIRED BY THE GUARANTEED TRANSACTION OF SECTION III-F, AND RELAY TELEPORTS, OVER EVERY SUITE OF TABLE III. THE TWO TRANSACTION COLUMNS ARE NESTED—*search* COVERS ALL NINE ROUTES PER CIRCUIT, OF WHICH THE *reported* ONE IS A MEMBER—WHILE RELAYS ARE TELEPORTS, COUNTED OVER THOSE SAME NINE ROUTES BUT NOT TIED TO A TRANSACTION.

Suite	Cores	Circuits	Transactions		Relays (search)
			Reported	Search	
25q	4	6	0	0	4
36q	4	6	0	0	5
64q	6	9	0	42	145
100q	6	2	6	73	250
200q	12	2	10	214	756
360q	20	2	66	745	3822
Total		27	82	1074	4982

creases its 64-qubit EPR geometric mean by 30.9%, but dsABRE still improves on TelesABRE by 33.4%.

The defaults of Table I are not tuned per instance. In the one-at-a-time sweeps of the supplementary material, every default is the best value swept on the 64-qubit suite and within 2.5% of the best at 25 qubits, and the advantage persists as the teleport-to-SWAP cost ratio varies from 10 to 100.

E. Comparison with *pytket-dqc*

This comparison must be interpreted as a cost-model study, not a like-for-like router comparison. *pytket-dqc* [6] uses hypergraph partitioning and gate teleportation, allowing one EPR pair to serve several gates when it remains live. We run its strongest default distributor that returns a result and report the best of five seeds. Its input network is more permissive than the evaluated device: all qubits are available for data and the number of live link qubits per core is unbounded.

Under those assumptions (Table VI), dsABRE uses 56.1% and 13.8% fewer EPR pairs on the 25- and 36-qubit suites; at 64 qubits, *pytket-dqc*’s 150.2 e-bits against dsABRE’s 173.3 EPR pairs is a 15.3% advantage for *pytket-dqc*.

TABLE VI

PYTKET-DQC AGAINST DSABRE, BY SUITE. *PYTKET-DQC* RUNS ITS DEFAULT COMPUTATION ON A NETWORK MORE PERMISSIVE THAN THE DEVICE (SECTION IV-E). SUITE FIGURES ARE GEOMETRIC MEANS OVER THE CIRCUITS OF TABLE III. *Unbuildable* COUNTS THE DISTRIBUTIONS THAT PROBABLY EXCEED THE DEVICE’S COMMUNICATION CAPACITY: EXACT AT 25 AND 36 QUBITS, A LOWER BOUND AT 64, WHERE MULTIPLIER’S CAPACITY SEARCH HAD NOT TERMINATED WHEN ITS BUDGET EXPIRED AND IS LEFT UNDETERMINED RATHER THAN COUNTED.

Suite	<i>pytket-dqc</i>	dsABRE	Δ (%)	unbuildable
	e-bits	EPR		
25 logical qubits	34.9	15.3	−56.1	4/6
36 logical qubits	12.2	10.5	−13.8	2/6
64 logical qubits	150.2	173.3	+15.3	6/9

However, 4/6, 2/6, and at least 6/9 of the respective distributions provably cannot be materialised within the evaluated communication-port capacity—25-qubit AE requires 20 simultaneous link qubits at a core where the device provides two.

Entanglement lifetime is a separate assumption. Re-scoring the same *pytket-dqc* distributions with a maximum of three gates per EPR pair changes the 64-qubit result to a 59.6% dsABRE advantage. The reversal shows where gate teleportation is valuable—circuits with long reusable interaction groups—but also that its reported EPR count depends on maintaining entanglement across roughly ten gates per pair. dsABRE’s teledata count is independent of that lifetime because each pair is consumed immediately.

Takeaway: The controlled TelesABRE comparison establishes the main result: capacity-aware teledata routing reduces EPR use and improves completion on the tested finite-capacity architectures. The *pytket-dqc* study identifies a complementary limitation—gate-teleportation amortisation can be preferable when long-lived entanglement and sufficient communication memory are available—and should not be read as a direct win or loss without those qualifications.

V. RELATED WORK

Distributed compilation combines qubit allocation, communication selection, routing, and scheduling. This section positions dsABRE at the routing stage; Section I gives the broader pipeline.

SABRE-style distributed routing. TelesABRE [10] is the closest prior router and the main experimental baseline. Both methods extend a SABRE-style, distance-and-lookahead search to multi-core hardware. TelesABRE selects from local SWAP and inter-core candidates in one pooled search and uses shortest paths on a contracted communication graph. dsABRE differs in three respects. It prices *availability*: a teledata move is scored as a macro-action including port staging and any required eviction, rather than as a bare hop an occupied port would delete. It *selects* by a priority rule, draining executable same-core front-layer gates first—under TelesABRE’s pooled arg min, 59.5% of its 64-qubit communication operations fire while a same-core front gate is still waiting. And it charges *capacity* against the landing core’s slack, backed by the legality floor of Eq. 4. The central difference is thus whether port availability and destination capacity are represented in

the candidate action and its cost. TeleSABRE additionally considers telegate operations.

DMapS [11] is a concurrent end-to-end approach that combines KaHyPar-based partitioning with a SABRE-derived router. Its cross-core primitive is a two-EPR remote SWAP, whereas dSABRE teleports one logical state into a layout-reserved free slot, and that choice propagates into a different device model. A remote SWAP exchanges two states across the link and a one-EPR remote CNOT moves nothing, so both leave each chip’s occupancy unchanged. DMapS therefore never needs a free slot: a chip may stay full for the whole route, and no capacity condition is maintained during routing. Its coupling graphs omit communication qubits accordingly, so every physical qubit carries data, an inter-chip edge weighs the same as an intra-chip one, and no remote operation can be blocked by an occupied port. These are modelling choices, not errors, but they leave the two EPR counts comparable only with the assumptions stated. Over the nine-circuit 64-qubit suite dSABRE uses $1.43\times$ fewer EPR pairs in geometric mean, DMapS winning only on GHZ and Graphstate, which KaHyPar partitions almost perfectly; the per-circuit head-to-head is in the online appendices.

Device model. The three comparisons differ in what they assume the hardware provides, and dSABRE’s assumptions are the most conservative. Following TeleSABRE, it represents communication ports explicitly, treats a port as an ordinary slot that must be evacuated before it can carry a state, gives an inter-core link a cost of its own, bounds every core’s capacity, and consumes each EPR pair at once. The other two relax exactly these points: `pytket-dqc` allows unboundedly many live link qubits per core and entanglement that survives a whole group of gates, and DMapS omits the ports and with them the capacity question. Modelling more of the device is not demanding more of it: dSABRE asks only for a connected coupling graph, the two to three ports per core the evaluated devices already have, two spare slots per core and one more on the device (Eqs. 12–13), and no entanglement storage at all, whereas the alternative models assume communication memory or occupancy-preserving remote operations the evaluated architecture does not represent. Extra ports, heterogeneous link costs, or a non-uniform core graph enter through the same distance tables and the same d_{prep} term, leaving the score and the completion argument unchanged.

Allocation, communication, and scheduling. Hypergraph-based methods assign qubits to cores by minimising cut communication [5], [6]. They are complementary to dSABRE: their output can supply an initial allocation, while dSABRE resolves the local movement and capacity constraints that arise during execution. Finally, time-aware DQC compilers schedule entanglement generation and remote operations subject to link and buffer constraints [17], [18]. Such scheduling is orthogonal to the present EPR-count objective and is a natural downstream extension of the teleport trace produced by dSABRE.

VI. FUTURE WORK

Telegate and communication bursts. Scoring a telegate is not the hard part, and it is not what stands in the way.

A telegate executes a remote gate without relocating either operand, so it consumes no destination capacity and enters Eq. 7 with $c_{\text{cap}} = 0$, keeping only the port staging d_{prep} its two link qubits need; its progress term is the front-layer gate it retires outright rather than a distance reduction. Implemented that way and allowed to compete with teledata, it costs 93.5% more EPR on the 64-qubit suite, and the best of seventeen scoring variants we swept is still 1.0% worse than teledata alone. Amortisation is the reason, and it is measurable: one teledata pair already serves 15.75 gates on this suite, 83.4% of relocations serving two or more, whereas an ungrouped telegate serves exactly one. A telegate therefore pays only in grouped form, for which the router must expose commutation-derived bursts, choose the beneficiary core, and hold entanglement for the group’s duration. AutoComm [19] and QuComm [20] give pre-processing directions, and lifetime-bounded grouping in [21] shows that a burst is worth taking only when its entanglement survives the hardware limit. The measurement is in the online appendices.

Scheduling, fidelity, and interconnect co-design. This paper minimises EPR consumption rather than makespan or output error, and the trace it emits is unscheduled. A scheduling pass should batch teleports whose links, ports, and staging operations do not conflict, and should account for entanglement-generation rate, storage lifetime, and hardware error rates; existing time-aware compilers [17], [18] provide starting points. The objective should stay explicitly multi-objective, since EPR count, circuit depth, and fidelity trade off in hardware-dependent ways. The interconnect is the other free variable. The evaluated devices give each pair of adjacent cores a single link and each core two or three ports, which is what the baseline architectures provide rather than a limit of the method: Section III-D already enumerates every link joining two cores as its own candidate, though on the evaluated devices that loop never has more than one link to weigh. Widening it should relieve the availability term d_{prep} first, since a second link turns an occupied port into an alternative rather than an obstacle, and should interact with the capacity mechanism, because a core with more ports has more ways to spend and to recover a free slot. Finding where the return on added links falls off against its hardware cost is a useful input to interconnect co-design [20], [22], [23].

VII. CONCLUSION

This paper presented dSABRE, a SABRE-style teledata router for multi-core distributed quantum computers. Its design principle is to treat an inter-core move as a complete macro-action rather than a local SWAP with a larger edge cost, scoring the routing progress it buys—front-layer distance reduction and DAG-depth-discounted lookahead—against the two costs specific to a directional teleport, port staging and destination capacity. Capacity enters twice, and the separation is the point: as a graded score term that ranks candidates, and as a hard legality floor that, with a bounded recovery transaction, makes dSABRE terminate having routed every gate under checkable pre-routing conditions.

On 21 MQT Bench circuits at 25–64 logical qubits dSABRE reduces geometric-mean EPR consumption by 49–62% against

TelesABRE under the shared teledata cost model, completes all 45 evaluated circuit–architecture instances to the baseline’s 40, and compiles a 360-qubit QFT on twenty cores in 48 s. Removing both capacity mechanisms costs 87.8% and leaves two instances unrouted, whereas replacing the BFS-layer extended set costs 12.4%: the principal advance is not the number of heuristic terms but the separation of resource-aware ranking from an explicit legality condition and a completion-preserving recovery. Recovery itself is a safeguard, not an optimisation—no reported route at 25–64 qubits invokes it, while at 100, 200, and 360 qubits it retires 6, 10, and 66 gates.

These reductions are in one resource, under one communication model. The direct comparison is with TelesABRE, which shares teledata and local-SWAP accounting; remote-SWAP and telegate-based tools need their own capacity and entanglement-lifetime assumptions stated before their counts can be read against ours. The emitted traces are also unscheduled, so fewer EPR pairs does not by itself imply a shorter makespan or a higher-fidelity result. Joint selection between teledata and grouped telegate operations, followed by time- and fidelity-aware scheduling of the resulting trace, is the natural next step.

Artefact availability: The dsABRE implementation, the routed-circuit benchmark suites, per-circuit JSON results, and the supplementary material are available at <https://github.com/ebony72/dsabre>.

ACKNOWLEDGMENTS

This work was partially supported by the Australian Research Council (Grant No. DP250102952 and DP220102059). The author used Anthropic’s Claude (via Claude Code) to assist with refactoring parts of the implementation, authoring benchmark scripts, drafting figure sources, and copy-editing. All AI-generated content was reviewed and verified by the author, who takes full responsibility for the correctness and originality of the work.

REFERENCES

- [1] D. Cuomo, M. Caleffi, and A. S. Cacciapuoti, “Towards a distributed quantum computing ecosystem,” *IET Quantum Communication*, vol. 1, no. 1, pp. 3–8, 2020.
- [2] L. J. Stephenson, D. P. Nadlinger, B. C. Nichol, S. An, P. Drmota, T. G. Ballance, K. Thirumalai, J. F. Goodwin, D. M. Lucas, and C. J. Ballance, “High-rate, high-fidelity entanglement of qubits across an elementary quantum network,” *Physical Review Letters*, vol. 124, no. 11, p. 110501, 2020.
- [3] S. Daiss, S. Langenfeld, S. Welte, E. Distant, P. Thomas, L. Hartung, O. Morin, and G. Rempe, “A quantum-logic gate between distant quantum-network modules,” *Science*, vol. 371, no. 6529, pp. 614–617, 2021.
- [4] P. Escofet, A. Ovide, M. Bandic, L. Prielinger, C. G. Almudever, S. Feld, E. Alarcón, and S. Abadal, “Revisiting the mapping of quantum circuits: Entering the multi-core era,” *ACM Transactions on Quantum Computing*, 2024, arXiv:2403.17205.
- [5] P. Andres-Martinez and C. Heunen, “Automated distribution of quantum circuits via hypergraph partitioning,” *Physical Review A*, vol. 100, no. 3, p. 032308, 2019.
- [6] P. Andres-Martinez, D. Mills, T. Forrer, and L. Henaut, “CQCL/pytket-dqc,” <https://github.com/CQCL/pytket-dqc>, Jun. 2024.
- [7] G. Li, Y. Ding, and Y. Xie, “Tackling the qubit mapping problem for NISQ-era quantum devices,” in *Proceedings of the 24th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2019, pp. 1001–1014.

- [8] Qiskit contributors, “Qiskit: An open-source framework for quantum computing,” 2024.
- [9] J. Eisert, K. Jacobs, P. Papadopoulos, and M. B. Plenio, “Optimal local implementation of nonlocal quantum gates,” *Physical Review A*, vol. 62, no. 5, p. 052317, 2000.
- [10] E. Russo, E. Vinciguerra, M. Palesi, D. Patti, G. Ascia, and V. Catania, “TelesABRE: Heuristic layout synthesis in multi-core quantum systems with teleport interconnect,” in *Proceedings of the IEEE International Conference on Quantum Computing and Engineering (QCE)*, 2025, pp. 749–758, arXiv:2505.08928. Code: <https://github.com/Haimrich/telesabre>.
- [11] T. Luo, Y. Zheng, Y. Deng, and X. Fu, “DMapS: End-to-end qubit mapping and routing for distributed quantum computing architectures,” *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 2025, code: <https://github.com/RoccoLoter/DMapS>.
- [12] A. Barenco, C. H. Bennett, R. Cleve, D. P. DiVincenzo, N. Margolus, P. Shor, T. Sleator, J. A. Smolin, and H. Weinfurter, “Elementary gates for quantum computation,” *Physical Review A*, vol. 52, no. 5, pp. 3457–3467, 1995.
- [13] C. H. Bennett, G. Brassard, C. Crépeau, R. Jozsa, A. Peres, and W. K. Wootters, “Teleporting an unknown quantum state via dual classical and Einstein-Podolsky-Rosen channels,” *Physical Review Letters*, vol. 70, no. 13, pp. 1895–1899, 1993.
- [14] H. Zou, M. Treinish, K. Hartman, A. Ivrii, and J. Lishman, “LightSABRE: A lightweight and enhanced SABRE algorithm,” 2024. [Online]. Available: <https://arxiv.org/abs/2409.08368>
- [15] P. Escofet, A. Ovide, C. G. Almudever, E. Alarcón, and S. Abadal, “Hungarian qubit assignment for optimized mapping of quantum circuits on multi-core architectures,” *IEEE Computer Architecture Letters*, vol. 22, no. 2, pp. 161–164, 2023.
- [16] N. Quetschlich, L. Burgholzer, and R. Wille, “MQT Bench: Benchmarking software and design automation tools for quantum computing,” *Quantum*, vol. 7, p. 1062, 2023.
- [17] J. M. Baker, C. Duckering, A. Hoover, and F. T. Chong, “Time-sliced quantum circuit partitioning for modular architectures,” in *Proceedings of the 17th ACM International Conference on Computing Frontiers (CF)*, 2020, pp. 98–107.
- [18] D. Ferrari, A. S. Cacciapuoti, M. Amoretti, and M. Caleffi, “Compiler design for distributed quantum computing,” *IEEE Transactions on Quantum Engineering*, vol. 2, pp. 1–20, 2021.
- [19] A. Wu, H. Zhang, G. Li, A. Shabani, Y. Xie, and Y. Ding, “AutoComm: A framework for enabling efficient communication in distributed quantum programs,” in *Proceedings of the 55th IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2022, arXiv:2207.11674.
- [20] A. Wu, Y. Ding, and A. Li, “QuComm: Optimizing collective communication for distributed quantum computing,” in *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2023.
- [21] M. Bandini, D. Ferrari, S. Carretta, and M. Amoretti, “Optimized compilation for distributed quantum computing,” 2026, arXiv:2602.24062.
- [22] P. Escofet, S. B. Rached, S. Rodrigo, C. G. Almudever, E. Alarcón, and S. Abadal, “Interconnect fabrics for multi-core quantum processors: A context analysis,” in *Proceedings of the 16th International Workshop on Network on Chip Architectures (NoCArc)*, 2023, arXiv:2309.07313.
- [23] J. Liu, A. Zang, M. Suchara, T. Zhong, and P. D. Hovland, “Hardware-software co-design for distributed quantum computing,” in *2025 62nd ACM/IEEE Design Automation Conference (DAC)*, 2025, pp. 1–6.



Sanjiang Li received his B.Sc. in mathematics from Shaanxi Normal University in 1996 and his Ph.D. in mathematics from Sichuan University in 2001. He is currently a professor at the Centre for Quantum Software and Information at the University of Technology Sydney (UTS). Prior to joining UTS, he worked in the Department of Computer Science and Technology at Tsinghua University from 2001 to 2008. His primary research interests include knowledge representation, artificial intelligence, and quantum circuit compilation.